TP – Instances d'un micro-service Spring Boot et Docker

Objectifs

- L'objectif de ce TP est de comprendre progressivement :
  - la création d'un micro-service Spring Boot ;
  - le rôle du port utilisé par un service ;
  - l'exécution de plusieurs instances d'un même micro-service ;
  - le problème lié à l'utilisation d'un même port ;
  - l'exécution de plusieurs instances sur des ports différents ;
  - la conteneurisation du micro-service avec Docker.

1 Créer un premier micro-service

- Créer une application Spring Boot contenant un contrôleur proposant l'URL :

  ```
  /hello
  ```

- Le service doit accepter un paramètre `name` et retourner :

  ```
  Hello <name>!
  ```

- Par exemple :

  ```
  http://localhost:8080/hello?name=Alice
  ```

  doit retourner :

  ```
  Hello Alice!
  ```

  Si aucun nom n'est fourni, le service retournera :

  ```
  Hello World!
  ```

Travail demandé

- Créer le contrôleur permettant d'obtenir ce comportement.
- Lancer ensuite l'application avec :

  ```
  mvn spring-boot:run
  ```

- Tester le service, par exemple avec :

  ```
  curl "http://localhost:8080/hello?name=Alex"
  ```

2 Lancer une deuxième instance sur le même port

- L'objectif est maintenant d'exécuter une deuxième instance du **même micro-service**.
- Dans un deuxième terminal, lancer à nouveau :

  ```
  mvn spring-boot:run
  ```

- Répondre aux questions suivantes :
  1. Que se passe-t-il ?
  2. Observer le message d'erreur.
  3. Quel est le problème rencontré ?
  4. Pourquoi la deuxième instance ne peut-elle pas démarrer ?

3 Lancer deux instances sur des ports différents

- Lancer maintenant une deuxième instance du même micro-service sur le port `8081`. Dans un deuxième terminal :

  ```
  mvn spring-boot:run -Dspring-boot.run.arguments="--server.port=8081"
  ```

- On dispose alors de deux instances :

  ```
  Instance 1 : localhost:8080
  Instance 2 : localhost:8081
  ```

- Tester les deux instances :

  ```
  curl "http://localhost:8080/hello?name=Alice"
  ```

  puis :

  ```
  curl "http://localhost:8081/hello?name=Bob"
  ```

- Répondre aux questions suivantes :
  1. Les deux instances exécutent-elles le même code ?
  2. Quelle est la différence entre un micro-service et une instance d'un micro-service ?
  3. Quel est le rôle du port dans cette architecture ?

4 Conteneuriser le micro-service

- L'objectif est maintenant d'exécuter le micro-service dans un conteneur Docker. Créer un fichier `Dockerfile` à la racine du projet.
- Construire l'image :

  ```
  docker build -t hello-service .
  ```

  Vérifier que l'image a été créée :

  ```
  docker images
  ```

- Lancer une première instance du micro-service dans un conteneur :

  ```
  docker run -p 8080:8080 hello-service
  ```

- Tester ensuite :

  ```
  curl "http://localhost:8080/hello?name=Alice"
  ```

5 Lancer plusieurs instances avec Docker

- Lancer maintenant une deuxième instance du même micro-service dans un autre conteneur :

  ```
  docker run -p 8081:8080 hello-service
  ```

  Tester :

  ```
  curl "http://localhost:8081/hello?name=Bob"
  ```

- Répondre aux questions suivantes :

  1. Observer les deux commandes :

     ```
     docker run -p 8080:8080 hello-service
     docker run -p 8081:8080 hello-service
     ```

  2. Que signifie `8080:8080` ?
  3. Que signifie `8081:8080` ?
  4. Pourquoi les deux conteneurs peuvent-ils utiliser le même port interne `8080` ?
  5. Quelle différence existe-t-il entre le port de la machine hôte et le port du conteneur ?